

```

--!native
-- SimpleSpy V3 (Mobile Optimized + Obsidian Red Theme)

if getgenv().SimpleSpyExecuted and type(getgenv().SimpleSpyShutdown)
== "function" then
    getgenv().SimpleSpyShutdown()
end

local realconfigs = {
    logcheckcaller = false,
    autoblock = false,
    funcEnabled = true,
    advancedinfo = false,
    supersecretdevtoggle = false
}

local configs = newproxy(true)
local configsmetatable = getmetatable(configs)
configsmetatable.__index = function(self, index) return
realconfigs[index] end

local oth = syn and syn.oth
local unhook = oth and oth.unhook
local hook = oth and oth.hook

local lower = string.lower
local byte = string.byte
local round = math.round
local running = coroutine.running
local resume = coroutine.resume
local status = coroutine.status
local yield = coroutine.yield
local create = coroutine.create
local close = coroutine.close
local OldDebugId = game.GetDebugId
local info = debug.info

local IsA = game.IsA
local tostring = tostring
local tonumber = tonumber
local delay = task.delay
local spawn = task.spawn
local clear = table.clear
local clone = table.clone

local function blankfunction(...) return ... end

local get_thread_identity = (syn and syn.get_thread_identity) or

```

```

getidentity or getthreadidentity
local set_thread_identity = (syn and syn.set_thread_identity) or
setidentity
local islclosure = islclosure or is_l_closure
local threadfuncs = (get_thread_identity and set_thread_identity and
true) or false

local getinfo = getinfo or blankfunction
local getupvalues = getupvalues or debug.getupvalues or blankfunction
local getconstants = getconstants or debug.getconstants or
blankfunction

local getcustomasset = getsynasset or getcustomasset
local getcallingscript = getcallingscript or blankfunction
local newcclosure = newcclosure or blankfunction
local clonefunction = clonefunction or blankfunction
local cloneref = cloneref or blankfunction
local request = request or syn and syn.request
local makewritable = makewritable or function(tbl)
setreadonly(tbl,false) end
local makereadonly = makereadonly or function(tbl)
setreadonly(tbl,true) end
local isreadonly = isreadonly or table.isfrozen

local setclipboard = setclipboard or toclipboard or set_clipboard or
(Clipboard and Clipboard.set) or function(...) return end

local hookmetamethod = hookmetamethod or (makewritable and
makereadonly and getrawmetatable) and function(obj: object,
metamethod: string, func: Function)
    local old = getrawmetatable(obj)
    if hookfunction then
        return hookfunction(old[metamethod],func)
    else
        local oldmetamethod = old[metamethod]
        makewritable(old)
        old[metamethod] = func
        makereadonly(old)
        return oldmetamethod
    end
end
end

local function Create(instance, properties, children)
    local obj = Instance.new(instance)
    for i, v in next, properties or {} do
        obj[i] = v
        for _, child in next, children or {} do child.Parent = obj end
    end
end

```

```

        return obj;
    end

    local function SafeGetService(service) return
    cloneref(game:GetService(service)) end

    local function Search(logtable,tbl)
        table.insert(logtable,tbl)
        for i,v in tbl do
            if type(v) == "table" then return table.find(logtable,v) ~=
            nil or Search(v) end
        end
    end

    local function IsCyclicTable(tbl)
        local checkedtables = {}
        local function SearchTable(tbl)
            table.insert(checkedtables,tbl)
            for i,v in next, tbl do
                if type(v) == "table" then return
                table.find(checkedtables,v) and true or SearchTable(v) end
            end
        end
        return SearchTable(tbl)
    end

    local function deepclone(args: table, copies: table): table
        local copy = nil
        copies = copies or {}
        if type(args) == 'table' then
            if copies[args] then
                copy = copies[args]
            else
                copy = {}
                copies[args] = copy
                for i, v in next, args do copy[deepclone(i, copies)] =
                deepclone(v, copies) end
            end
        elseif typeof(args) == "Instance" then
            copy = cloneref(args)
        else
            copy = args
        end
        return copy
    end

    local function rawtostring(userdata)
        if type(userdata) == "table" or typeof(userdata) == "userdata"

```

```

then
    local rawmetatable = getrawmetatable(userdata)
    local cachedstring = rawmetatable and rawget(rawmetatable,
"__tostring")
    if cachedstring then
        local wasreadonly = isreadonly(rawmetatable)
        if wasreadonly then makewritable(rawmetatable) end
        rawset(rawmetatable, "__tostring", nil)
        local safestring = tostring(userdata)
        rawset(rawmetatable, "__tostring", cachedstring)
        if wasreadonly then makereadonly(rawmetatable) end
        return safestring
    end
end
return tostring(userdata)
end

local CoreGui = SafeGetService("CoreGui")
local Players = SafeGetService("Players")
local RunService = SafeGetService("RunService")
local UserInputService = SafeGetService("UserInputService")
local TweenService = SafeGetService("TweenService")
local TextService = SafeGetService("TextService")
local http = SafeGetService("HttpService")
local GuiInset = game:GetService("GuiService"):GetGuiInset()

local function jsone(str) return http.JSONEncode(str) end
local function jsond(str)
    local suc,err = pcall(http.JSONDecode,http,str)
    return suc and err or suc
end

function ErrorPrompt(Message,state) warn(Message) end

local Highlight = (isfile and loadfile and isfile("Highlight.lua") and
loadfile("Highlight.lua")()) or
loadstring(game:HttpGet("https://raw.githubusercontent.com/78n/SimpleS
py/main/Highlight.lua"))()
local LazyFix =
loadstring(game:HttpGet("https://raw.githubusercontent.com/78n/Roblox/
refs/heads/main/Lua/Libraries/DataToCode/DataToCode.lua"))()

-----
-----
-- MODERN RED THEME UI
-----
-----
local Theme = {

```

```

MainBG = Color3.fromRGB(15, 15, 18),
PanelBG = Color3.fromRGB(20, 20, 25),
CodeBG = Color3.fromRGB(12, 12, 15),
TopBar = Color3.fromRGB(15, 15, 18),
Accent = Color3.fromRGB(230, 40, 40),
ButtonBG = Color3.fromRGB(32, 32, 38),
ButtonHover = Color3.fromRGB(45, 45, 55),
TextDark = Color3.fromRGB(180, 180, 190),
TextLight = Color3.fromRGB(240, 240, 245),
CornerRadius = UDim.new(0, 6)
}

local SimpleSpy3 = Create("ScreenGui",{ResetOnSpawn = false, Name =
"SimpleSpyMobile"})
local Storage = Create("Folder",{})

-- CENTERED POSITION FOR MOBILE (0.5, -225, 0.5, -134)
local Background = Create("Frame",{Parent =
SimpleSpy3,BackgroundColor3 = Theme.MainBG,BackgroundTransparency =
0,BorderSizePixel = 0,Position = UDim2.new(0.5, -225, 0.5, -134),Size
= UDim2.new(0, 450, 0, 268)})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent =
Background})
Create("UIStroke", {Color = Theme.Accent, Thickness = 1.5,
Transparency = 0.5, Parent = Background})

local LeftPanel = Create("Frame",{Parent = Background,BackgroundColor3
= Theme.PanelBG,BorderSizePixel = 0,Position = UDim2.new(0, 0, 0,
19),Size = UDim2.new(0, 131, 0, 249)})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent =
LeftPanel})

local LogList = Create("ScrollingFrame",{Parent = LeftPanel,Active =
true,BackgroundColor3 = Color3.new(1, 1, 1),BackgroundTransparency =
1,BorderSizePixel = 0,Position = UDim2.new(0, 0, 0, 9),Size =
UDim2.new(0, 131, 0, 232),CanvasSize = UDim2.new(0, 0, 0,
0),ScrollBarThickness = 2, ScrollBarImageColor3 = Theme.Accent})
local UIListLayout = Create("UIListLayout",{Parent =
LogList,HorizontalAlignment =
Enum.HorizontalAlignment.Center,SortOrder =
Enum.SortOrder.LayoutOrder, Padding = UDim.new(0, 2)})

local RightPanel = Create("Frame",{Parent =
Background,BackgroundColor3 = Theme.PanelBG,BorderSizePixel =
0,Position = UDim2.new(0, 131, 0, 19),Size = UDim2.new(0, 319, 0,
249)})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent =
RightPanel})

```

```
local CodeBox = Create("Frame",{Parent = RightPanel,BackgroundColor3 = Theme.CodeBG,BorderSizePixel = 0,Size = UDim2.new(0, 319, 0, 119)})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent = CodeBox})
```

```
local ScrollingFrame = Create("ScrollingFrame",{Parent = RightPanel,Active = true,BackgroundColor3 = Color3.new(1, 1, 1),BackgroundTransparency = 1,Position = UDim2.new(0, 0, 0.5, 0),Size = UDim2.new(1, 0, 0.5, -9),CanvasSize = UDim2.new(0, 0, 0, 0),ScrollBarThickness = 2, ScrollBarImageColor3 = Theme.Accent})
local UIGridLayout = Create("UIGridLayout",{Parent = ScrollingFrame,HorizontalAlignment = Enum.HorizontalAlignment.Center,SortOrder = Enum.SortOrder.LayoutOrder,CellPadding = UDim2.new(0, 6, 0, 6),CellSize = UDim2.new(0, 94, 0, 27)})
```

```
local TopBar = Create("Frame",{Parent = Background,BackgroundColor3 = Theme.TopBar,BorderSizePixel = 0,Size = UDim2.new(0, 450, 0, 19)})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent = TopBar})
Create("Frame", {Parent = TopBar, BackgroundColor3 = Theme.TopBar, BorderSizePixel = 0, Position = UDim2.new(0,0,1,-6), Size = UDim2.new(1,0,0,6)})
```

```
local Simple = Create("TextButton",{Parent = TopBar,BackgroundColor3 = Color3.new(1, 1, 1),AutoButtonColor = false,BackgroundTransparency = 1,Position = UDim2.new(0, 10, 0, 0),Size = UDim2.new(0, 57, 0, 18),Font = Enum.Font.GothamBold,Text = "SimpleSpy",TextColor3 = Theme.TextLight,TextSize = 12,TextXAlignment = Enum.TextXAlignment.Left})
```

```
local CloseButton = Create("TextButton",{Parent = TopBar,BackgroundColor3 = Theme.TopBar,BorderSizePixel = 0,Position = UDim2.new(1, -19, 0, 0),Size = UDim2.new(0, 19, 0, 19),Font = Enum.Font.SourceSans,Text = "",TextColor3 = Color3.new(0, 0, 0),TextSize = 14})
```

```
local ImageLabel = Create("ImageLabel",{Parent = CloseButton,BackgroundColor3 = Color3.new(1, 1, 1),BackgroundTransparency = 1,Position = UDim2.new(0, 5, 0, 5),Size = UDim2.new(0, 9, 0, 9),Image = "http://www.roblox.com/asset/?id=5597086202", ImageColor3 = Theme.TextDark})
```

```
local MaximizeButton = Create("TextButton",{Parent = TopBar,BackgroundColor3 = Theme.TopBar,BorderSizePixel = 0,Position = UDim2.new(1, -38, 0, 0),Size = UDim2.new(0, 19, 0, 19),Font = Enum.Font.SourceSans,Text = "",TextColor3 = Color3.new(0, 0, 0),TextSize = 14})
```

```
local ImageLabel_2 = Create("ImageLabel",{Parent =
```

```

MaximizeButton,BackgroundColor3 = Color3.new(1, 1,
1),BackgroundTransparency = 1,Position = UDim2.new(0, 5, 0, 5),Size =
UDim2.new(0, 9, 0, 9),Image =
"http://www.roblox.com/asset/?id=5597108117", ImageColor3 =
Theme.TextDark})
local MinimizeButton = Create("TextButton",{Parent =
TopBar,BackgroundColor3 = Theme.TopBar,BorderSizePixel = 0,Position =
UDim2.new(1, -57, 0, 0),Size = UDim2.new(0, 19, 0, 19),Font =
Enum.Font.SourceSans,Text = "",TextColor3 = Color3.new(0, 0,
0),TextSize = 14})
local ImageLabel_3 = Create("ImageLabel",{Parent =
MinimizeButton,BackgroundColor3 = Color3.new(1, 1,
1),BackgroundTransparency = 1,Position = UDim2.new(0, 5, 0, 5),Size =
UDim2.new(0, 9, 0, 9),Image =
"http://www.roblox.com/asset/?id=5597105827", ImageColor3 =
Theme.TextDark})

local ToolTip = Create("Frame",{Parent = SimpleSpy3,BackgroundColor3 =
Theme.MainBG,BackgroundTransparency = 0.05,BorderColor3 =
Color3.new(1, 1, 1),BorderSizePixel = 0,Size = UDim2.new(0, 200, 0,
50),ZIndex = 5,Visible = false})
Create("UICorner", {CornerRadius = Theme.CornerRadius, Parent =
ToolTip})
Create("UIStroke", {Color = Theme.Accent, Thickness = 1, Parent =
ToolTip})
local TextLabel = Create("TextLabel",{Parent =
ToolTip,BackgroundColor3 = Color3.new(1, 1, 1),BackgroundTransparency
= 1,Position = UDim2.new(0, 6, 0, 6),Size = UDim2.new(0, 188, 0,
38),ZIndex = 5,Font = Enum.Font.GothamMedium,Text = "This is some
slightly longer text.",TextColor3 = Theme.TextLight,TextSize =
12,TextWrapped = true,TextXAlignment =
Enum.TextXAlignment.Left,TextYAlignment = Enum.TextYAlignment.Top})

-----
-----
local selectedColor = Color3.new(0.321569, 0.333333, 1)
local deselectedColor = Color3.new(0.8, 0.8, 0.8)
local layoutOrderNum = 999999999
local mainClosing = false
local closed = false
local sideClosing = false
local sideClosed = false
local maximized = false
local logs = {}
local selected = nil
local blacklist = {}
local blocklist = {}
local getNil = false

```

```

local connectedRemotes = {}
local toggle = false
local prevTables = {}
local remoteLogs = {}
getgenv().SIMPLESPYCONFIG_MaxRemotes = 300
local indent = 4
local scheduled = {}
local schedulerconnect
local SimpleSpy = {}
local topstr = ""
local bottomstr = ""
local remotesFadeIn
local rightFadeIn
local codebox
local p
local getnilrequired = false
local history = {}
local excluding = {}
local mouseInGui = false
local connections = {}
local DecompiledScripts = {}
local generation = {}
local running_threads = {}
local originalnamecall

local remoteEvent = Instance.new("RemoteEvent",Storage)
local unreliableRemoteEvent = Instance.new("UnreliableRemoteEvent")
local remoteFunction = Instance.new("RemoteFunction",Storage)
local NamecallHandler = Instance.new("BindableEvent",Storage)
local IndexHandler = Instance.new("BindableEvent",Storage)
local GetDebugIdHandler = Instance.new("BindableFunction",Storage)

local originalEvent = remoteEvent.FireServer
local originalUnreliableEvent = unreliableRemoteEvent.FireServer
local originalFunction = remoteFunction.InvokeServer
local GetDebugIDInvoke = GetDebugIdHandler.Invoke

function GetDebugIdHandler.OnInvoke(obj: Instance) return
OldDebugId(obj) end
local function ThreadGetDebugId(obj: Instance): string return
GetDebugIDInvoke(GetDebugIdHandler,obj) end

local synv3 = false
if syn and identifyexecutor then
    local _, version = identifyexecutor()
    if (version and version:sub(1, 2) == 'v3') then synv3 = true end
end

```

```

xpcall(function()
    if isfile and readfile and isfolder and makefolder then
        local cachedconfigs = isfile("SimpleSpy//Settings.json") and
jsond(readfile("SimpleSpy//Settings.json"))
        if cachedconfigs then
            for i,v in next, realconfigs do
                if cachedconfigs[i] == nil then cachedconfigs[i] = v
end
            end
            realconfigs = cachedconfigs
        end
        if not isfolder("SimpleSpy") then makefolder("SimpleSpy") end
        if not isfolder("SimpleSpy//Assets") then
makefolder("SimpleSpy//Assets") end
        if not isfile("SimpleSpy//Settings.json") then
writefile("SimpleSpy//Settings.json",jsone(realconfigs)) end
        configsmetatable.__newindex = function(self,index,newindex)
            realconfigs[index] = newindex
            writefile("SimpleSpy//Settings.json",jsone(realconfigs))
        end
    else
        configsmetatable.__newindex = function(self,index,newindex)
realconfigs[index] = newindex end
    end
end,function(err) end)

local function logthread(thread: thread)
table.insert(running_threads,thread) end

function clean()
    local max = getenv().SIMPLESPYCONFIG_MaxRemotes
    if not typeof(max) == "number" and math.floor(max) ~= max then max
= 500 end
    if #remoteLogs > max then
        for i = 100, #remoteLogs do
            local v = remoteLogs[i]
            if typeof(v[1]) == "RBXScriptConnection" then
v[1]:Disconnect() end
            if typeof(v[2]) == "Instance" then v[2]:Destroy() end
        end
        local newLogs = {}
        for i = 1, 100 do table.insert(newLogs, remoteLogs[i]) end
        remoteLogs = newLogs
    end
end

local function ThreadIsNotDead(thread: thread): boolean return not
status(thread) == "dead" end

```

```

function scaleToolTip()
    local size = TextService.GetTextSize(TextLabel.Text,
TextLabel.TextSize, TextLabel.Font, Vector2.new(196, math.huge))
    TextLabel.Size = UDim2.new(0, size.X, 0, size.Y)
    ToolTip.Size = UDim2.new(0, size.X + 4, 0, size.Y + 4)
end

function onToggleButtonHover()
    if not toggle then TweenService:Create(Simple, TweenInfo.new(0.5),
{TextColor3 = Theme.Accent}):Play()
    else TweenService:Create(Simple, TweenInfo.new(0.5), {TextColor3 =
Color3.fromRGB(68, 206, 91)}):Play() end
end

function onToggleButtonUnhover() TweenService:Create(Simple,
TweenInfo.new(0.5), {TextColor3 = Theme.TextLight}):Play() end
function onXButtonHover() TweenService:Create(CloseButton,
TweenInfo.new(0.2), {BackgroundColor3 = Theme.Accent}):Play() end
function onXButtonUnhover() TweenService:Create(CloseButton,
TweenInfo.new(0.2), {BackgroundColor3 = Theme.TopBar}):Play() end

function onToggleButtonClick()
    if toggle then TweenService:Create(Simple, TweenInfo.new(0.5),
{TextColor3 = Theme.Accent}):Play()
    else TweenService:Create(Simple, TweenInfo.new(0.5), {TextColor3 =
Color3.fromRGB(68, 206, 91)}):Play() end
    toggleSpyMethod()
end

function connectResize()
    if not workspace.CurrentCamera then
workspace:GetPropertyChangedSignal("CurrentCamera"):Wait() end
    local lastCam =
workspace.CurrentCamera:GetPropertyChangedSignal("ViewportSize"):Conne
ct(bringBackOnResize)

workspace:GetPropertyChangedSignal("CurrentCamera"):Connect(function()
    lastCam:Disconnect()
    if typeof(lastCam) == 'Connection' then lastCam:Disconnect()
end

    lastCam =
workspace.CurrentCamera:GetPropertyChangedSignal("ViewportSize"):Conne
ct(bringBackOnResize)
    end)
end

function bringBackOnResize()

```

```

    validateSize()
    if sideClosed then minimizeSize() else maximizeSize() end
    local currentX = Background.AbsolutePosition.X
    local currentY = Background.AbsolutePosition.Y
    local viewportSize = workspace.CurrentCamera.ViewportSize
    if (currentX < 0) or (currentX > (viewportSize.X - (sideClosed and
131 or Background.AbsoluteSize.X))) then
        if currentX < 0 then currentX = 0 else currentX =
viewportSize.X - (sideClosed and 131 or Background.AbsoluteSize.X) end
    end
    if (currentY < 0) or (currentY > (viewportSize.Y - (closed and 19
or Background.AbsoluteSize.Y) - GuiInset.Y)) then
        if currentY < 0 then currentY = 0 else currentY =
viewportSize.Y - (closed and 19 or Background.AbsoluteSize.Y) -
GuiInset.Y end
    end
    TweenService.Create(TweenService, Background, TweenInfo.new(0.1),
{Position = UDim2.new(0, currentX, 0, currentY)}):Play()
end

function onBarInput(input)
    -- MOBILE TOUCH FIX
    if input.UserInputType == Enum.UserInputType.MouseButton1 or
input.UserInputType == Enum.UserInputType.Touch then
        local lastPos = UserInputService.GetMouseLocation()
        local mainPos = Background.AbsolutePosition
        local offset = mainPos - lastPos
        local currentPos = offset + lastPos
        if not connections["drag"] then
            connections["drag"] =
RunService.RenderStepped:Connect(function()
                local newPos = UserInputService.GetMouseLocation()
                if newPos ~= lastPos then
                    local currentX = (offset + newPos).X
                    local currentY = (offset + newPos).Y
                    local viewportSize =
workspace.CurrentCamera.ViewportSize
                    if (currentX < 0 and currentX < currentPos.X) or
(currentX > (viewportSize.X - (sideClosed and 131 or
TopBar.AbsoluteSize.X)) and currentX > currentPos.X) then
                        if currentX < 0 then currentX = 0 else
currentX = viewportSize.X - (sideClosed and 131 or
TopBar.AbsoluteSize.X) end
                    end
                    if (currentY < 0 and currentY < currentPos.Y) or
(currentY > (viewportSize.Y - (closed and 19 or
Background.AbsoluteSize.Y) - GuiInset.Y) and currentY > currentPos.Y)
then

```

```

        if currentY < 0 then currentY = 0 else
currentY = viewportSize.Y - (closed and 19 or
Background.AbsoluteSize.Y) - GuiInset.Y end
        end
        currentPos = Vector2.new(currentX, currentY)
        lastPos = newPos
        TweenService.Create(TweenService, Background,
TweenInfo.new(0.1), {Position = UDim2.new(0, currentPos.X, 0,
currentPos.Y)}):Play()
        end
    end)
end
    table.insert(connections,
UserInputService.InputEnded:Connect(function(inputE)
    if input == inputE then
        if connections["drag"] then
connections["drag"]:Disconnect(); connections["drag"] = nil end
        end
    end))
end
end

function fadeOut(elements)
    local data = {}
    for _, v in next, elements do
        if typeof(v) == "Instance" and v:IsA("GuiObject") and
v.Visible then
            spawn(function()
                data[v] = {BackgroundTransparency =
v.BackgroundTransparency}
                TweenService.Create(v, TweenInfo.new(0.5),
{BackgroundTransparency = 1}):Play()
                if v:IsA("TextBox") or v:IsA("TextButton") or
v:IsA("TextLabel") then
                    data[v].TextTransparency = v.TextTransparency
                    TweenService.Create(v, TweenInfo.new(0.5),
{TextTransparency = 1}):Play()
                elseif v:IsA("ImageButton") or v:IsA("ImageLabel")
then
                    data[v].ImageTransparency = v.ImageTransparency
                    TweenService.Create(v, TweenInfo.new(0.5),
{ImageTransparency = 1}):Play()
                end
                delay(0.5,function()
                    v.Visible = false
                    for i, x in next, data[v] do v[i] = x end
                    data[v] = true
                end)
            end)
        end
    end
end

```

```

        end)
    end
end
return function()
    for i, _ in next, data do
        spawn(function()
            local properties = {BackgroundTransparency =
i.BackgroundTransparency}
            i.BackgroundTransparency = 1
            TweenService:Create(i, TweenInfo.new(0.5),
{BackgroundTransparency = properties.BackgroundTransparency}):Play()
            if i:IsA("TextBox") or i:IsA("TextButton") or
i:IsA("TextLabel") then
                properties.TextTransparency = i.TextTransparency
                i.TextTransparency = 1
                TweenService:Create(i, TweenInfo.new(0.5),
{TextTransparency = properties.TextTransparency}):Play()
            elseif i:IsA("ImageButton") or i:IsA("ImageLabel")
then
                properties.ImageTransparency = i.ImageTransparency
                i.ImageTransparency = 1
                TweenService:Create(i, TweenInfo.new(0.5),
{ImageTransparency = properties.ImageTransparency}):Play()
            end
            i.Visible = true
        end)
    end
end
end

function toggleMinimize(override)
    if mainClosing and not override or maximized then return end
    mainClosing = true; closed = not closed
    if closed then
        if not sideClosed then toggleSideTray(true) end
        LeftPanel.Visible = true
        remotesFadeIn = fadeOut(LeftPanel:GetDescendants())
        TweenService:Create(LeftPanel, TweenInfo.new(0.5), {Size =
UDim2.new(0, 131, 0, 0)}):Play()
        wait(0.5)
    else
        TweenService:Create(LeftPanel, TweenInfo.new(0.5), {Size =
UDim2.new(0, 131, 0, 249)}):Play()
        wait(0.5)
        if remotesFadeIn then remotesFadeIn(); remotesFadeIn = nil end
        bringBackOnResize()
    end
    mainClosing = false
end

```

```

end

function toggleSideTray(override)
    if sideClosing and not override or maximized then return end
    sideClosing = true; sideClosed = not sideClosed
    if sideClosed then
        rightFadeIn = fadeOut(RightPanel:GetDescendants())
        wait(0.5); minimizeSize(0.5); wait(0.5); RightPanel.Visible =
false
    else
        if closed then toggleMinimize(true) end
        RightPanel.Visible = true
        maximizeSize(0.5); wait(0.5)
        if rightFadeIn then rightFadeIn() end
        bringBackOnResize()
    end
    sideClosing = false
end

function toggleMaximize()
    if not sideClosed and not maximized then
        maximized = true
        local disable = Instance.new("TextButton")
        local prevSize = UDim2.new(0, CodeBox.AbsoluteSize.X, 0,
CodeBox.AbsoluteSize.Y)
        local prevPos = UDim2.new(0, CodeBox.AbsolutePosition.X, 0,
CodeBox.AbsolutePosition.Y)
        disable.Size = UDim2.new(1, 0, 1, 0)
        disable.BackgroundColor3 = Color3.new()
        disable.BorderSizePixel = 0
        disable.Text = 0
        disable.ZIndex = 3
        disable.BackgroundTransparency = 1
        disable.AutoButtonColor = false
        CodeBox.ZIndex = 4
        CodeBox.Position = prevPos
        CodeBox.Size = prevSize
        TweenService.Create(CodeBox, TweenInfo.new(0.5), {Size =
UDim2.new(0.5, 0, 0.5, 0), Position = UDim2.new(0.25, 0, 0.25,
0)}) : Play()
        TweenService.Create(disable, TweenInfo.new(0.5),
{BackgroundTransparency = 0.5}) : Play()
        disable.MouseButton1Click:Connect(function()
            if UserInputService.GetMouseLocation().Y + GuiInset.Y >=
CodeBox.AbsolutePosition.Y and UserInputService.GetMouseLocation().Y +
GuiInset.Y <= CodeBox.AbsolutePosition.Y + CodeBox.AbsoluteSize.Y and
UserInputService.GetMouseLocation().X >= CodeBox.AbsolutePosition.X
and UserInputService.GetMouseLocation().X <=

```

```

CodeBox.AbsolutePosition.X + CodeBox.AbsoluteSize.X then return end
    TweenService:Create(CodeBox, TweenInfo.new(0.5), {Size =
prevSize, Position = prevPos}):Play()
    TweenService:Create(disable, TweenInfo.new(0.5),
{BackgroundTransparency = 1}):Play()
        wait(0.5)
        disable:Destroy()
        CodeBox.Size = UDim2.new(1, 0, 0.5, 0)
        CodeBox.Position = UDim2.new(0, 0, 0, 0)
        CodeBox.ZIndex = 0
        maximized = false
    end)
end
end

function isInResizeRange(p)
    local relativeP = p - Background.AbsolutePosition
    local range = 5
    if relativeP.X >= TopBar.AbsoluteSize.X - range and relativeP.Y >=
Background.AbsoluteSize.Y - range and relativeP.X <=
TopBar.AbsoluteSize.X and relativeP.Y <= Background.AbsoluteSize.Y
then
        return true, 'B'
    elseif relativeP.X >= TopBar.AbsoluteSize.X - range and
relativeP.X <= Background.AbsoluteSize.X then
        return true, 'X'
    elseif relativeP.Y >= Background.AbsoluteSize.Y - range and
relativeP.Y <= Background.AbsoluteSize.Y then
        return true, 'Y'
    end
    return false
end

function isInDragRange(p)
    local relativeP = p - Background.AbsolutePosition
    local topbarAS = TopBar.AbsoluteSize
    return relativeP.X <= topbarAS.X - CloseButton.AbsoluteSize.X * 3
and relativeP.X >= 0 and relativeP.Y <= topbarAS.Y and relativeP.Y >=
0 or false
end

function mouseEntered() end
function mouseMoved() end

function maximizeSize(speed)
    if not speed then speed = 0.05 end
    TweenService:Create(LeftPanel, TweenInfo.new(speed), { Size =
UDim2.fromOffset(LeftPanel.AbsoluteSize.X, Background.AbsoluteSize.Y -

```

```

TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(RightPanel, TweenInfo.new(speed), { Size =
UDim2.fromOffset(Background.AbsoluteSize.X - LeftPanel.AbsoluteSize.X,
Background.AbsoluteSize.Y - TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(TopBar, TweenInfo.new(speed), { Size =
UDim2.fromOffset(Background.AbsoluteSize.X, TopBar.AbsoluteSize.Y)
}):Play()
    TweenService:Create(ScrollingFrame, TweenInfo.new(speed), { Size =
UDim2.fromOffset(Background.AbsoluteSize.X - LeftPanel.AbsoluteSize.X,
110), Position = UDim2.fromOffset(0, Background.AbsoluteSize.Y - 119 -
TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(CodeBox, TweenInfo.new(speed), { Size =
UDim2.fromOffset(Background.AbsoluteSize.X - LeftPanel.AbsoluteSize.X,
Background.AbsoluteSize.Y - 119 - TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(LogList, TweenInfo.new(speed), { Size =
UDim2.fromOffset(LogList.AbsoluteSize.X, Background.AbsoluteSize.Y -
TopBar.AbsoluteSize.Y - 18) }):Play()
end

```

```

function minimizeSize(speed)
    if not speed then speed = 0.05 end
    TweenService:Create(LeftPanel, TweenInfo.new(speed), { Size =
UDim2.fromOffset(LeftPanel.AbsoluteSize.X, Background.AbsoluteSize.Y -
TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(RightPanel, TweenInfo.new(speed), { Size =
UDim2.fromOffset(0, Background.AbsoluteSize.Y - TopBar.AbsoluteSize.Y)
}):Play()
    TweenService:Create(TopBar, TweenInfo.new(speed), { Size =
UDim2.fromOffset(LeftPanel.AbsoluteSize.X, TopBar.AbsoluteSize.Y)
}):Play()
    TweenService:Create(ScrollingFrame, TweenInfo.new(speed), { Size =
UDim2.fromOffset(0, 119), Position = UDim2.fromOffset(0,
Background.AbsoluteSize.Y - 119 - TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(CodeBox, TweenInfo.new(speed), { Size =
UDim2.fromOffset(0, Background.AbsoluteSize.Y - 119 -
TopBar.AbsoluteSize.Y) }):Play()
    TweenService:Create(LogList, TweenInfo.new(speed), { Size =
UDim2.fromOffset(LogList.AbsoluteSize.X, Background.AbsoluteSize.Y -
TopBar.AbsoluteSize.Y - 18) }):Play()
end

```

```

function validateSize()
    local x, y = Background.AbsoluteSize.X, Background.AbsoluteSize.Y
    local screenSize = workspace.CurrentCamera.ViewportSize
    if x + Background.AbsolutePosition.X > screenSize.X then
        x = screenSize.X - Background.AbsolutePosition.X >= 450 and
screenSize.X - Background.AbsolutePosition.X or 450
    elseif y + Background.AbsolutePosition.Y > screenSize.Y then

```

```

        y = screenSize.Y - Background.AbsolutePosition.Y >= 268 and
screenSize.Y - Background.AbsolutePosition.Y or 268
    end
    Background.Size = UDim2.fromOffset(x, y)
end

function backgroundUserInput(input)
    local mousePos = UserInputService.GetMouseLocation() - GuiInset
    local inResizeRange, type = isInResizeRange(mousePos)
    -- MOBILE TOUCH FIX
    if (input.UserInputType == Enum.UserInputType.MouseButton1 or
input.UserInputType == Enum.UserInputType.Touch) and inResizeRange
then
        local lastPos = UserInputService.GetMouseLocation()
        local offset = Background.AbsoluteSize - lastPos
        local currentPos = lastPos + offset
        if not connections["SIMPLESPY_RESIZE"] then
            connections["SIMPLESPY_RESIZE"] =
RunService.RenderStepped:Connect(function()
                local newPos = UserInputService.GetMouseLocation()
                if newPos ~= lastPos then
                    local currentX = (newPos + offset).X
                    local currentY = (newPos + offset).Y
                    if currentX < 450 then currentX = 450 end
                    if currentY < 268 then currentY = 268 end
                    currentPos = Vector2.new(currentX, currentY)
                    Background.Size = UDim2.fromOffset((not sideClosed
and not closed and (type == "X" or type == "B")) and currentPos.X or
Background.AbsoluteSize.X, (not closed and (type == "Y" or type ==
"B")) and currentPos.Y or Background.AbsoluteSize.Y)
                    validateSize()
                    if sideClosed then minimizeSize() else
maximizeSize() end
                    lastPos = newPos
                end
            end)
        end
        table.insert(connections,
UserInputService.InputEnded:Connect(function(inputE)
            if input == inputE then
                if connections["SIMPLESPY_RESIZE"] then
connections["SIMPLESPY_RESIZE"]:Disconnect();
connections["SIMPLESPY_RESIZE"] = nil end
                end
            end))
        elseif isInDragRange(mousePos) then
            onBarInput(input)
        end
    end
end

```

```

end

function eventSelect(frame)
    if selected and selected.Log then
        if selected.Button then spawn(function()
TweenService:Create(selected.Button, TweenInfo.new(0.5),
{BackgroundColor3 = Theme.ButtonBG}):Play() end) end
        selected = nil
    end
    for _, v in next, logs do if frame == v.Log then selected = v end
end
    if selected and selected.Log then
        spawn(function() TweenService:Create(frame.Button,
TweenInfo.new(0.5), {BackgroundColor3 = Color3.fromRGB(60, 20,
25}):Play() end)
        codebox:setRaw(selected.GenScript)
    end
    if sideClosed then toggleSideTray() end
end

function updateFunctionCanvas() ScrollingFrame.CanvasSize =
UDim2.fromOffset(UIGridLayout.AbsoluteContentSize.X,
UILayout.AbsoluteContentSize.Y) end
function updateRemoteCanvas() LogList.CanvasSize =
UDim2.fromOffset(UIListLayout.AbsoluteContentSize.X,
UIListLayout.AbsoluteContentSize.Y) end

function makeToolTip(enable, text)
    if enable and text then
        if ToolTip.Visible then
            ToolTip.Visible = false
            if connections["ToolTip"] then
connections["ToolTip"]:Disconnect() end
        end
        local first = true
        connections["ToolTip"] =
RunService.RenderStepped:Connect(function()
            local MousePos = UserInputService.GetMouseLocation()
            local topLeft = MousePos + Vector2.new(20, -15)
            local bottomRight = topLeft + ToolTip.AbsoluteSize
            local ViewportSize = workspace.CurrentCamera.ViewportSize
            if topLeft.X < 0 then topLeft = Vector2.new(0, topLeft.Y)
            elseif bottomRight.X > ViewportSize.X then topLeft =
Vector2.new(ViewportSize.X - ToolTip.AbsoluteSize.X, topLeft.Y) end
            if topLeft.Y < 0 then topLeft = Vector2.new(topLeft.X, 0)
            elseif bottomRight.Y > ViewportSize.Y - 35 then topLeft =
Vector2.new(topLeft.X, ViewportSize.Y - ToolTip.AbsoluteSize.Y - 35)
        end
    end
end

```

```

        if topLeft.X <= MousePos.X and topLeft.Y <= MousePos.Y
then topLeft = Vector2.new(MousePos.X - ToolTip.AbsoluteSize.X - 2,
MousePos.Y - ToolTip.AbsoluteSize.Y - 2) end
        if first then ToolTip.Position =
UDim2.fromOffset(topLeft.X, topLeft.Y); first = false
        else ToolTip.TweenPosition(UDim2.fromOffset(topLeft.X,
topLeft.Y), "Out", "Linear", 0.1) end
        end)
        TextLabel.Text = text; TextLabel.TextScaled = true;
ToolTip.Visible = true
        else
            if ToolTip.Visible then ToolTip.Visible = false; if
connections["ToolTip"] then connections["ToolTip"]:Disconnect() end
            end
        end
    end
end

function newButton(name, description, onClick)
    local FunctionTemplate = Create("Frame", {Name =
"FunctionTemplate", Parent = ScrollingFrame, BackgroundColor3 =
Color3.new(1, 1, 1), BackgroundTransparency = 1, Size = UDim2.new(0, 94,
0, 27)})
    local Button = Create("TextButton", {Name = "Button", Parent =
FunctionTemplate, BackgroundColor3 =
Theme.ButtonBG, BackgroundTransparency = 0, BorderSizePixel = 0, Position
= UDim2.new(0, 2, 0, 2), Size = UDim2.new(0, 90, 0, 23), AutoButtonColor
= false, Font = Enum.Font.SourceSans, Text = "", TextColor3 =
Color3.new(0, 0, 0), TextSize = 14})
    Create("UICorner", {CornerRadius = UDim.new(0, 4), Parent =
Button})
    local stroke = Create("UIStroke", {Color = Theme.Accent, Thickness
= 1, Transparency = 0.8, Parent = Button})
    local ColorBar = Create("Frame", {Name = "ColorBar", Parent =
FunctionTemplate, BackgroundColor3 = Theme.Accent, BorderSizePixel =
0, Position = UDim2.new(0, 8, 0, 8), Size = UDim2.new(0, 3, 0,
11), ZIndex = 3})
    Create("UICorner", {CornerRadius = UDim.new(1, 0), Parent =
ColorBar})
    local Text = Create("TextLabel", {Text = name, Name = "Text", Parent
= FunctionTemplate, BackgroundColor3 = Color3.new(1, 1,
1), BackgroundTransparency = 1, Position = UDim2.new(0, 16, 0, 2), Size =
UDim2.new(0, 72, 0, 23), ZIndex = 2, Font =
Enum.Font.GothamMedium, TextColor3 = Theme.TextLight, TextSize =
11, TextXAlignment = Enum.TextXAlignment.Left})

    Button.MouseEnter:Connect(function()
        TweenService:Create(Button, TweenInfo.new(0.2),
{BackgroundColor3 = Theme.ButtonHover}):Play()
    end)
end

```

```

        TweenService:Create(stroke, TweenInfo.new(0.2), {Transparency
= 0}):Play()
        makeToolTip(true, description())
    end)
    Button.MouseLeave:Connect(function()
        TweenService:Create(Button, TweenInfo.new(0.2),
{BackgroundColor3 = Theme.ButtonBG}):Play()
        TweenService:Create(stroke, TweenInfo.new(0.2), {Transparency
= 0.8}):Play()
        makeToolTip(false)
    end)
    FunctionTemplate.AncestryChanged:Connect(function()
makeToolTip(false) end)
    Button.MouseButton1Click:Connect(function(...)
logthread(running()); onClick(FunctionTemplate, ...) end)
    updateFunctionCanvas()
end

function newRemote(type, data)
    if layoutOrderNum < 1 then layoutOrderNum = 999999999 end
    local remote = data.remote
    local callingscript = data.callingscript
    local accentColor = (type == "event" and Color3.fromRGB(255, 60,
60)) or Color3.fromRGB(150, 60, 255)

    local RemoteTemplate = Create("Frame",{LayoutOrder =
layoutOrderNum,Name = "RemoteTemplate",Parent =
LogList,BackgroundColor3 = Color3.new(1, 1, 1),BackgroundTransparency
= 1,Size = UDim2.new(0, 117, 0, 30)})
    local Button = Create("TextButton",{Name = "Button",Parent =
RemoteTemplate,BackgroundColor3 =
Theme.ButtonBG,BackgroundTransparency = 0,BorderSizePixel = 0,Position
= UDim2.new(0, 4, 0, 2),Size = UDim2.new(0, 115, 0,
26),AutoButtonColor = false,Font = Enum.Font.SourceSans,Text =
"",TextColor3 = Color3.new(0, 0, 0),TextSize = 14})
    Create("UICorner", {CornerRadius = UDim.new(0, 4), Parent =
Button})
    Create("UIStroke", {Color = accentColor, Thickness = 1,
Transparency = 0.8, Parent = Button})
    local ColorBar = Create("Frame",{Name = "ColorBar",Parent =
RemoteTemplate,BackgroundColor3 = accentColor,BorderSizePixel =
0,Position = UDim2.new(0, 10, 0, 9),Size = UDim2.new(0, 3, 0,
12),ZIndex = 2})
    Create("UICorner", {CornerRadius = UDim.new(1, 0), Parent =
ColorBar})
    local Text = Create("TextLabel",{TextTruncate =
Enum.TextTruncate.AtEnd,Name = "Text",Parent =
RemoteTemplate,BackgroundColor3 = Color3.new(1, 1,

```

```
1),BackgroundTransparency = 1,Position = UDim2.new(0, 18, 0, 2),Size =
UDim2.new(0, 95, 0, 26),ZIndex = 2,Font = Enum.Font.GothamMedium,Text
= remote.Name,TextColor3 = Theme.TextLight,TextSize =
12,TextXAlignment = Enum.TextXAlignment.Left})
```

```
    local log = {
        Name = remote.name, Function = data.infofunc or "--Function
Info is disabled",
        Remote = remote, DebugId = data.id, metamethod =
data.metamethod, args = data.args,
        Log = RemoteTemplate, Button = Button, Blocked = data.blocked,
        Source = callingscript, returnvalue = data.returnvalue,
        GenScript = "-- Generating, please wait...\n-- (If this
message persists, the remote args are likely extremely long)"
    }
```

```
    logs[#logs + 1] = log
    local connect = Button.MouseButton1Click:Connect(function()
        logthread(running())
        eventSelect(RemoteTemplate)
        log.GenScript = genScript(log.Remote, log.args)
        if blocked then log.GenScript = "-- THIS REMOTE WAS PREVENTED
FROM FIRING TO THE SERVER BY SIMPLESPY\n\n" .. log.GenScript end
        if selected == log and RemoteTemplate then
            eventSelect(RemoteTemplate) end
        end)
    layoutOrderNum -= 1
    table.insert(remoteLogs, 1, {connect, RemoteTemplate})
    clean()
    updateRemoteCanvas()
end
```

```
function genScript(remote, args)
    prevTables = {}
    local gen = ""
    if #args > 0 then
        xpcall(function()
            gen = "local args = "..LazyFix.Convert(args, true) .. "\n"
        end,function(err)
            gen ..= "-- An error has occurred:\n--"..err.."
TableToString failure! Reverting to legacy functionality (results may
vary)\nlocal args = {"
            xpcall(function()
                for i, v in next, args do
                    if type(i) ~= "Instance" and type(i) ~= "userdata"
then gen = gen .. "\n    [object] = "
                        elseif type(i) == "string" then gen = gen .. '\n
["' .. i .. '"] = '

```

```

elseif type(i) == "userdata" and typeof(i) ~=
"Instance" then gen = gen .. "\n    [" .. string.format("nil
--[[[%s]]", typeof(v)) .. "]] = "
elseif type(i) == "userdata" then gen = gen .. "\n
[game." .. i:GetFullName() .. "]] = " end
if type(v) ~= "Instance" and type(v) ~= "userdata"
then gen = gen .. "object"
elseif type(v) == "string" then gen = gen .. "' '
.. v .. "' '
elseif type(v) == "userdata" and typeof(v) ~=
"Instance" then gen = gen .. string.format("nil --[[[%s]]", typeof(v))
elseif type(v) == "userdata" then gen = gen ..
"game." .. v:GetFullName() end
end
gen ..= "\n}\n\n"
end,function() gen ..= "}\n-- Legacy tableToString
failure! Unable to decompile." end)
end)
if not remote:IsDescendantOf(game) and not getnilrequired then
gen = "function getNil(name,class) for _,v in next,
getnilinstances()do if v.ClassName==class and v.Name==name then return
v;end end end\n\n" .. gen
end
if remote:IsA("RemoteEvent") or
remote:IsA("UnreliableRemoteEvent") then gen ..=
LazyFix.ConvertKnown("Instance", remote) ..
":FireServer(unpack(args))"
elseif remote:IsA("RemoteFunction") then gen = gen ..
LazyFix.ConvertKnown("Instance", remote) ..
":InvokeServer(unpack(args))" end
else
if remote:IsA("RemoteEvent") or
remote:IsA("UnreliableRemoteEvent") then gen ..=
LazyFix.ConvertKnown("Instance", remote) .. ":FireServer()"
elseif remote:IsA("RemoteFunction") then gen ..=
LazyFix.ConvertKnown("Instance", remote) .. ":InvokeServer()" end
end
prevTables = {}
return gen
end

local CustomGeneration = {
Vector3 = (function() local temp = {} for i,v in Vector3 do if
type(v) == "vector" then temp[v] = `Vector3.{i}` end end return temp
end)(),
Vector2 = (function() local temp = {} for i,v in Vector2 do if
type(v) == "userdata" then temp[v] = `Vector2.{i}` end end return temp
end)(),

```

```

    CFrame = { [CFrame.identity] = "CFrame.identity" }
}

local number_table = { ["inf"] = "math.huge", ["-inf"] = "-math.huge",
["nan"] = "0/0" }

local ufunctions
ufunctions = {
    TweenInfo = function(u) return `TweenInfo.new({u.Time},
{u.EasingStyle}, {u.EasingDirection}, {u.RepeatCount}, {u.Reverses},
{u.DelayTime})` end,
    Ray = function(u) return
`Ray.new({ufunctions["Vector3"](u.Origin)},
{ufunctions["Vector3"](u.Direction)})` end,
    BrickColor = function(u) return `BrickColor.new({u.Number})` end,
    NumberRange = function(u) return `NumberRange.new({u.Min},
{u.Max})` end,
    Region3 = function(u) local center = u.CFrame.Position; local
centersize = u.Size/2; return
`Region3.new({ufunctions["Vector3"](center-centersize)},
{ufunctions["Vector3"](center+centersize)})` end,
    Faces = function(u) local faces = {}; if u.Top then
table.insert(faces, "Top") end; if u.Bottom then table.insert(faces,
"Enum.NormalId.Bottom") end; if u.Left then table.insert(faces,
"Enum.NormalId.Left") end; if u.Right then table.insert(faces,
"Enum.NormalId.Right") end; if u.Back then table.insert(faces,
"Enum.NormalId.Back") end; if u.Front then table.insert(faces,
"Enum.NormalId.Front") end; return `Faces.new({table.concat(faces, "
")})` end,
    EnumItem = function(u) return tostring(u) end,
    Enums = function(u) return "Enum" end,
    Enum = function(u) return `Enum.{u}` end,
    Vector3 = function(u) return CustomGeneration.Vector3[u] or
`Vector3.new({u})` end,
    Vector2 = function(u) return CustomGeneration.Vector2[u] or
`Vector2.new({u})` end,
    CFrame = function(u) return CustomGeneration.CFrame[u] or
`CFrame.new({table.concat({u:GetComponents()},", ")})` end,
    PathWaypoint = function(u) return
`PathWaypoint.new({ufunctions["Vector3"](u.Position)}, {u.Action},
"{u.Label}")` end,
    UDim = function(u) return `UDim.new({u})` end,
    UDim2 = function(u) return `UDim2.new({u})` end,
    Rect = function(u) return
`Rect.new({ufunctions["Vector2"](u.Min)},
{ufunctions["Vector2"](u.Max)})` end,
    Color3 = function(u) return `Color3.new({u.R}, {u.G}, {u.B})` end,
    RBXScriptSignal = function(u) return "RBXScriptSignal

```

```

--[[RBXScriptSignal's are not supported]]" end,
    RBXScriptConnection = function(u) return "RBXScriptConnection
--[[RBXScriptConnection's are not supported]]" end,
}

local typeofv2sfunctions = {
    number = function(v) local number = tostring(v); return
number_table[number] or number end,
    boolean = function(v) return tostring(v) end,
    string = function(v,l) return formatstr(v, l) end,
    ["function"] = function(v) return f2s(v) end,
    table = function(v, l, p, n, vtv, i, pt, path, tables, tI) return
t2s(v, l, p, n, vtv, i, pt, path, tables, tI) end,
    Instance = function(v) return i2p(v,generation[OldDebugId(v)])
end,
    userdata = function(v) if configs.advancedinfo then if
getrawmetatable(v) then return "newproxy(true)" end return
"newproxy(false)" end return "newproxy(true)" end
}

local typev2sfunctions = {
    userdata = function(v,vtypeof) if ufunctions[vtypeof] then return
ufunctions[vtypeof](v) end return `{vtypeof}({rawtostring(v)})
--[[Generation Failure]]` end,
    vector = ufunctions["Vector3"]
}

function v2s(v, l, p, n, vtv, i, pt, path, tables, tI)
    local vtypeof = typeof(v)
    local vtypeoffunc = typeofv2sfunctions[vtypeof]
    local vtypefunc = typev2sfunctions[type(v)]
    if not tI then tI = {0} else tI[1] += 1 end
    if vtypeoffunc then return vtypeoffunc(v, l, p, n, vtv, i, pt,
path, tables, tI)
    elseif vtypefunc then return vtypefunc(v,vtypeof) end
    return `{vtypeof}({rawtostring(v)}) --[[Generation Failure]]`
end

function v2v(t)
    topstr = ""; bottomstr = ""; getnilrequired = false
    local ret = ""; local count = 1
    for i, v in next, t do
        if type(i) == "string" and i:match("^[%a_]+[%w_]*$") then ret
= ret .. "local " .. i .. " = " .. v2s(v, nil, nil, i, true) .. "\n"
        elseif rawtostring(i):match("^[%a_]+[%w_]*$") then ret = ret
.. "local " .. lower(rawtostring(i)) .. "_" .. rawtostring(count) .. "
= " .. v2s(v, nil, nil, lower(rawtostring(i)) .. "_" ..
rawtostring(count), true) .. "\n"

```

```

        else ret = ret .. "local " .. type(v) .. "_" ..
rawtostring(count) .. " = " .. v2s(v, nil, nil, type(v) .. "_" ..
rawtostring(count), true) .. "\n" end
        count = count + 1
    end
    if getnilrequired then topstr = "function getNil(name,class) for
_,v in next, getnilinstances() do if v.ClassName==class and
v.Name==name then return v;end end end\n" .. topstr end
    if #topstr > 0 then ret = topstr .. "\n" .. ret end
    if #bottomstr > 0 then ret = ret .. bottomstr end
    return ret
end

function tabletostring(tbl: table,format: boolean) end

function t2s(t, l, p, n, vtv, i, pt, path, tables, tI)
    local globalIndex = table.find(getenv(), t)
    if type(globalIndex) == "string" then return globalIndex end
    if not tI then tI = {0} end
    if not path then path = "" end
    if not l then l = 0; tables = {} end
    if not p then p = t end
    for _, v in next, tables do
        if n and rawequal(v, t) then
            bottomstr = bottomstr .. "\n" .. rawtostring(n) ..
rawtostring(path) .. " = " .. rawtostring(n) .. rawtostring(({v2p(v,
p)})[2])
            return "{} --[[DUPLICATE]]"
        end
    end
    table.insert(tables, t)
    local s = "{"
    local size = 0
    l += indent
    for k, v in next, t do
        size = size + 1
        if size > (getenv().SimpleSpyMaxTableSize or 1000) then s = s
.. "\n" .. string.rep(" ", l) .. "-- MAXIMUM TABLE SIZE REACHED";
break end
        if rawequal(k, t) then
            bottomstr ..= ` \n{n}{path} [{n}{path}] = {(rawequal(v,k)
and ` {n}{path}` or v2s(v, l, p, n, vtv, k, t, `{path} [{n}{path}]`,
tables))}`
            size -= 1; continue
        end
        local currentPath = ""
        if type(k) == "string" and k:match("^[%a_]+[%w_]*$") then
currentPath = "." .. k else currentPath = "[" .. v2s(k, l, p, n, vtv,

```

```

k, t, path .. currentPath, tables, tI) .. "]" end
    if size % 100 == 0 then scheduleWait() end
    s = s .. "\n" .. string.rep(" ", 1) .. "[" .. v2s(k, 1, p, n,
vtv, k, t, path .. currentPath, tables, tI) .. "]" = " .. v2s(v, 1, p,
n, vtv, k, t, path .. currentPath, tables, tI) .. ","
    end
    if #s > 1 then s = s:sub(1, #s - 1) end
    if size > 0 then s = s .. "\n" .. string.rep(" ", 1 - indent) end
    return s .. "}"
end

```

```

function f2s(f)
    for k, x in next, getgenv() do
        local isgucci, gpath
        if rawequal(x, f) then isgucci, gpath = true, "" elseif
type(x) == "table" then isgucci, gpath = v2p(f, x) end
        if isgucci and type(k) ~= "function" then
            if type(k) == "string" and k:match("^[%a_]+[%w_]*$") then
return k .. gpath else return "getgenv()[" .. v2s(k) .. "]" .. gpath
end
            end
        end
        if configs.funcEnabled then
            local funcname = info(f, "n")
            if funcname and funcname:match("^[%a_]+[%w_]*$") then return
`function {funcname}() end -- Function Called: {funcname}` end
            end
        return tostring(f)
    end
end

```

```

function i2p(i, customgen)
    if customgen then return customgen end
    local player = getplayer(i)
    local parent = i
    local out = ""
    if parent == nil then return "nil"
elseif player then
        while true do
            if parent and parent == player.Character then
                if player == Players.LocalPlayer then return
'game:GetService("Players").LocalPlayer.Character' .. out
                else return i2p(player) .. ".Character" .. out end
            else
                if parent.Name:match("[%a_]+[%w+]*") ~= parent.Name
then out = ':FindFirstChild(' .. formatstr(parent.Name) .. ')' .. out
                else out = "." .. parent.Name .. out end
            end
            task.wait(); parent = parent.Parent
        end
    end
end

```

```

        end
    elseif parent ~= game then
        while true do
            if parent and parent.Parent == game then
                if SafeGetService(parent.ClassName) then
                    if lower(parent.ClassName) == "workspace" then
return `workspace{out}` else return 'game:GetService("' ..
parent.ClassName .. '")' .. out end
                    else
                        if parent.Name:match("[%a_]+[%w_]*") then return
"game." .. parent.Name .. out else return 'game:FindFirstChild(' ..
formatstr(parent.Name) .. ')"' .. out end
                        end
                    elseif not parent.Parent then
                        getnilrequired = true; return 'getNil(' ..
formatstr(parent.Name) .. ', "' .. parent.ClassName .. '")' .. out
                        else
                            if parent.Name:match("[%a_]+[%w_]*") ~= parent.Name
then out = ':WaitForChild(' .. formatstr(parent.Name) .. ')"' .. out
else out = ':WaitForChild("' .. parent.Name .. '")' .. out end
                            end
                            if i:IsDescendantOf(Players.LocalPlayer) then return
'game:GetService("Players").LocalPlayer' .. out end
                            parent = parent.Parent; task.wait()
                        end
                    else return "game" end
                end
            end
        end

function getplayer(instance)
    for _, v in next, Players:GetPlayers() do if v.Character and
(instance:IsDescendantOf(v.Character) or instance == v.Character) then
return v end end
end

function v2p(x, t, path, prev)
    if not path then path = "" end
    if not prev then prev = {} end
    if rawequal(x, t) then return true, "" end
    for i, v in next, t do
        if rawequal(v, x) then
            if type(i) == "string" and i:match("^[%a_]+[%w_]*$") then
return true, (path .. "." .. i) else return true, (path .. "[" ..
v2s(i) .. "]"") end
            end
            if type(v) == "table" then
                local duplicate = false
                for _, y in next, prev do if rawequal(y, v) then duplicate
= true end end
            end
        end
    end
end

```

```

        if not duplicate then
            table.insert(prev, t)
            local found, p = v2p(x, v, path, prev)
            if found then
                if type(i) == "string" and
i:match("^[%a_]+[%w_]*$") then return true, "." .. i .. p else return
true, "[" .. v2s(i) .. "]" .. p end
            end
        end
    end
end
return false, ""
end

```

```

function formatstr(s, indentation)
    if not indentation then indentation = 0 end
    local handled, reachedMax = handlespecials(s, indentation)
    return "'" .. handled .. "'" .. (reachedMax and " --[[ MAXIMUM
STRING SIZE REACHED ]]" or "")
end

```

```

local function isFinished(coroutines: table)
    for _, v in next, coroutines do if status(v) == "running" then
return false end end
    return true
end

```

```

local specialstrings = {
    ["\n"] = function(thread,index) resume(thread,index,"\\n") end,
    ["\t"] = function(thread,index) resume(thread,index,"\\t") end,
    ["\\"] = function(thread,index) resume(thread,index,"\\\\") end,
    ["'"] = function(thread,index) resume(thread,index,"\\'") end
}

```

```

function handlespecials(s, indentation)
    local i = 0; local n = 1; local coroutines = {}
    local coroutineFunc = function(i, r) s = s:sub(0, i - 1) .. r ..
s:sub(i + 1, -1) end
    local timeout = 0
    repeat
        i += 1
        if timeout >= 10 then task.wait(); timeout = 0 end
        local char = s:sub(i, i)
        if byte(char) then
            timeout += 1
            local c = create(coroutineFunc)
            table.insert(coroutines, c)
            local specialfunc = specialstrings[char]

```

```

        if specialfunc then specialfunc(c,i); i += 1
        elseif byte(char) > 126 or byte(char) < 32 then resume(c,
i, "\\\" .. byte(char)); i += #rawtostring(byte(char)) end
        if i >= n * 100 then
            local extra = string.format('\" ..\n%s"', string.rep("
", indentation + indent))
            s = s:sub(0, i) .. extra .. s:sub(i + 1, -1)
            i += #extra; n += 1
        end
    end
until char == "" or i > (getgenv().SimpleSpyMaxStringSize or
10000)
while not isFinished(coroutines) do RunService.Heartbeat:Wait()
end
clear(coroutines)
if i > (getgenv().SimpleSpyMaxStringSize or 10000) then s =
string.sub(s, 0, getgenv().SimpleSpyMaxStringSize or 10000); return s,
true end
return s, false
end

```

```

function getScriptFromSrc(src)
    local realPath; local runningTest; local s, e; local match = false
    if src:sub(1, 1) == "=" then realPath = game; s = 2
    else
        runningTest = src:sub(2, e and e - 1 or -1)
        for _, v in next, getnilinstances() do if v.Name ==
runningTest then realPath = v; break end end
        s = #runningTest + 1
    end
    if realPath then
        e = src:sub(s, -1):find("%.")
        local i = 0
        repeat
            i += 1
            if not e then
                runningTest = src:sub(s, -1)
                local test = realPath.FindFirstChild(realPath,
runningTest)
                if test then realPath = test end
                match = true
            else
                runningTest = src:sub(s, e)
                local test = realPath.FindFirstChild(realPath,
runningTest)
                local yeOld = e
                if test then realPath = test; s = e + 2; e = src:sub(e
+ 2, -1):find("%."); e = e and e + yeOld or e

```

```

                else e = src:sub(e + 2, -1):find("%."); e = e and e +
yeOld or e end
            end
        until match or i >= 50
        end
        return realPath
    end

function schedule(f, ...) table.insert(scheduled, {f, ...}) end
function scheduleWait() local thread = running(); schedule(function()
resume(thread) end); yield() end

local function taskscheduler()
    if not toggle then scheduled = {}; return end
    if #scheduled > SIMPLESPYCONFIG_MaxRemotes + 100 then
table.remove(scheduled, #scheduled) end
    if #scheduled > 0 then
        local currentf = scheduled[1]; table.remove(scheduled, 1)
        if type(currentf) == "table" and type(currentf[1]) ==
"function" then pcall(unpack(currentf)) end
    end
end

local function tablecheck(tabletocheck,instance,id) return
tabletocheck[id] or tabletocheck[instance.Name] end

function remoteHandler(data)
    if configs.autoblock then
        local id = data.id
        if excluding[id] then return end
        if not history[id] then history[id] = {badOccurrences = 0,
lastCall = tick()} end
        if tick() - history[id].lastCall < 1 then
history[id].badOccurrences += 1; return else history[id].badOccurrences
= 0 end
        if history[id].badOccurrences > 3 then excluding[id] = true;
return end
        history[id].lastCall = tick()
    end
    if (data.remote:IsA("RemoteEvent") or
data.remote:IsA("UnreliableRemoteEvent")) and lower(data.method) ==
"fireserver" then newRemote("event", data)
    elseif data.remote:IsA("RemoteFunction") and lower(data.method) ==
"invokeserver" then newRemote("function", data) end
end

local newindex = function(method,originalfunction,...)
    if typeof(...) == 'Instance' then

```

```

        local remote = cloneref(...)
        if remote:IsA("RemoteEvent") or remote:IsA("RemoteFunction")
or remote:IsA("UnreliableRemoteEvent") then
            if not configs.logcheckcaller and checkcaller() then
return originalfunction(...) end
                local id = ThreadGetDebugId(remote)
                local blockcheck = tablecheck(blocklist,remote,id)
                local args = {select(2,...)}
                if not tablecheck(blocklist,remote,id) and not
IsCyclicTable(args) then
                    local data = { method = method, remote = remote, args
= deepclone(args), infofunc = infofunc, callingscript = callingscript,
metamethod = "__index", blockcheck = blockcheck, id = id, returnvalue
= {} }

                    args = nil
                    if configs.funcEnabled then data.infofunc =
info(2,"f"); local calling = getcallingscript(); data.callingscript =
calling and cloneref(calling) or nil end
                    schedule(remoteHandler,data)
                end
                if blockcheck then return end
            end
        end
        return originalfunction(...)
    end

local newnamecall = newcclosure(function(...)
    local method = getnamecallmethod()
    if method and (method == "FireServer" or method == "fireServer" or
method == "InvokeServer" or method == "invokeServer") then
        if typeof(...) == 'Instance' then
            local remote = cloneref(...)
            if IsA(remote,"RemoteEvent") or
IsA(remote,"RemoteFunction") or IsA(remote,"UnreliableRemoteEvent")
then
                if not configs.logcheckcaller and checkcaller() then
return originalnamecall(...) end
                    local id = ThreadGetDebugId(remote)
                    local blockcheck = tablecheck(blocklist,remote,id)
                    local args = {select(2,...)}
                    if not tablecheck(blocklist,remote,id) and not
IsCyclicTable(args) then
                        local data = { method = method, remote = remote,
args = deepclone(args), infofunc = infofunc, callingscript =
callingscript, metamethod = "__namecall", blockcheck = blockcheck, id
= id, returnvalue = {} }

                        args = nil
                        if configs.funcEnabled then data.infofunc =

```

```

info(2,"f"); local calling = getcallingscript(); data.callingscript =
calling and cloneref(calling) or nil end
        schedule(remoteHandler,data)
    end
    if blockcheck then return end
end
end
    return originalnamecall(...)
end)

local newFireServer = newcclosure(function(...) return
newindex("FireServer",originalEvent,...) end)
local newUnreliableFireServer = newcclosure(function(...) return
newindex("FireServer",originalUnreliableEvent,...) end)
local newInvokeServer = newcclosure(function(...) return
newindex("InvokeServer",originalFunction,...) end)

local function disablehooks()
    if synv3 then
        unhook(getrawmetatable(game).__namecall,originalnamecall)
        unhook(Instance.new("RemoteEvent").FireServer, originalEvent)
        unhook(Instance.new("RemoteFunction").InvokeServer,
originalFunction)
        unhook(Instance.new("UnreliableRemoteEvent").FireServer,
originalUnreliableEvent)
        restorefunction(originalnamecall);
        restorefunction(originalEvent); restorefunction(originalFunction)
    else
        if hookmetamethod then
hookmetamethod(game,"__namecall",originalnamecall) else
hookfunction(getrawmetatable(game).__namecall,originalnamecall) end
            hookfunction(Instance.new("RemoteEvent").FireServer,
originalEvent)
            hookfunction(Instance.new("RemoteFunction").InvokeServer,
originalFunction)
            hookfunction(Instance.new("UnreliableRemoteEvent").FireServer,
originalUnreliableEvent)
        end
    end
end

function toggleSpy()
    if not toggle then
        local oldnamecall
        if synv3 then
            oldnamecall =
hook(getrawmetatable(game).__namecall,clonefunction(newnamecall))
            originalEvent =

```

```

hook(Instance.new("RemoteEvent").FireServer,
clonefunction(newFireServer))
    originalFunction =
hook(Instance.new("RemoteFunction").InvokeServer,
clonefunction(newInvokeServer))
    originalUnreliableEvent =
hook(Instance.new("UnreliableRemoteEvent").FireServer,
clonefunction(newUnreliableFireServer))
    else
        if hookmetamethod then oldnamecall = hookmetamethod(game,
"__namecall", clonefunction(newnamecall)) else oldnamecall =
hookfunction(getrawmetatable(game).__namecall,clonefunction(newnamecal
l)) end
        originalEvent =
hookfunction(Instance.new("RemoteEvent").FireServer,
clonefunction(newFireServer))
        originalFunction =
hookfunction(Instance.new("RemoteFunction").InvokeServer,
clonefunction(newInvokeServer))
        originalUnreliableEvent =
hookfunction(Instance.new("UnreliableRemoteEvent").FireServer,
clonefunction(newUnreliableFireServer))
    end
    originalnamecall = originalnamecall or function(...) return
oldnamecall(...) end
    else
        disablehooks()
    end
end
end

function toggleSpyMethod() toggleSpy(); toggle = not toggle end

local function shutdown()
    if schedulerconnect then schedulerconnect:Disconnect() end
    for _, connection in next, connections do connection:Disconnect()
end
    for i,v in next, running_threads do if ThreadIsNotDead(v) then
close(v) end end
    clear(running_threads); clear(connections); clear(logs);
clear(remoteLogs)
    disablehooks()
SimpleSpy3:Destroy()
Storage:Destroy()
UserInputService.MouseIconEnabled = true
getenv().SimpleSpyExecuted = false
end

if not getenv().SimpleSpyExecuted then

```

```

    local succeeded,err = pcall(function()
        if not RunService:IsClient() then error("SimpleSpy cannot run
on the server!") end
        getgenv().SimpleSpyShutdown = shutdown
        onToggleButtonClick()
        if not hookmetamethod then ErrorPrompt("Simple Spy V3 will not
function to it's fullest capability due to your executor not supporting
hookmetamethod.",true) end
        codebox = Highlight.new(CodeBox)
        logthread(spawn(function()
            local suc,err =
pcall(game.HttpGet,game,"https://raw.githubusercontent.com/78n/SimpleS
py/main/UpdateLog.lua")
            codebox:setRaw((suc and err) or "")
        end))
        getgenv().SimpleSpy = SimpleSpy
        getgenv().getNil = function(name,class) for _,v in next,
getnilinstances() do if v.ClassName == class and v.Name == name then
return v; end end end

        Background.MouseEnter:Connect(function(...) mouseInGui = true;
mouseEntered() end)
        Background.MouseLeave:Connect(function(...) mouseInGui =
false; mouseEntered() end)

TextLabel:GetPropertyChangedSignal("Text"):Connect(scaleToolTip)
MinimizeButton.MouseButton1Click:Connect(toggleMinimize)
MaximizeButton.MouseButton1Click:Connect(toggleSideTray)
Simple.MouseButton1Click:Connect(onToggleButtonClick)
CloseButton.MouseEnter:Connect(onXButtonHover)
CloseButton.MouseLeave:Connect(onXButtonUnhover)
Simple.MouseEnter:Connect(onToggleButtonHover)
Simple.MouseLeave:Connect(onToggleButtonUnhover)
CloseButton.MouseButton1Click:Connect(shutdown)
table.insert(connections,
UserInputService.InputBegan:Connect(backgroundUserInput))
connectResize()
SimpleSpy3.Enabled = true
logthread(spawn(function() delay(1,onToggleButtonUnhover)
end))

schedulerconnect = RunService.Heartbeat:Connect(taskscheduler)
bringBackOnResize()

-- MOBILE UI FALLBACK
local targetParent = (gethui and gethui()) or (syn and
syn.protect_gui and syn.protect_gui(SimpleSpy3)) or CoreGui
local successParent, errParent = pcall(function()
SimpleSpy3.Parent = targetParent end)

```

```

        if not successParent then SimpleSpy3.Parent =
Players.LocalPlayer:WaitForChild("PlayerGui") end

        logthread(spawn(function()
            local lp = Players.LocalPlayer or
Players:GetPropertyChangedSignal("LocalPlayer"):Wait() or
Players.LocalPlayer
            generation = { [OldDebugId(lp)] =
'game:GetService("Players").LocalPlayer', [OldDebugId(lp:GetMouse())]
= 'game:GetService("Players").LocalPlayer:GetMouse',
[OldDebugId(game)] = "game", [OldDebugId(workspace)] = "workspace" }
            end))
        end)
        if succeeded then getgenv().SimpleSpyExecuted = true else
shutdown(); ErrorPrompt("An error has occurred:\n"..rawtostring(err));
return end
        else
            SimpleSpy3:Destroy()
            return
        end
end

function SimpleSpy:newButton(name, description, onClick) return
newButton(name, description, onClick) end

newButton("Copy Code", function() return "Click to copy code" end,
function()
    setclipboard(codebox.getString()); TextLabel.Text = "Copied
successfully!"
end)

newButton("Copy Remote", function() return "Click to copy the path of
the remote" end, function()
    if selected and selected.Remote then
setclipboard(v2s(selected.Remote)); TextLabel.Text = "Copied!" end
end)

newButton("Run Code", function() return "Click to execute code" end,
function()
    local Remote = selected and selected.Remote
    if Remote then
        TextLabel.Text = "Executing..."
        xpcall(function()
            local returnvalue
            if Remote:IsA("RemoteEvent") or
Remote:IsA("UnreliableRemoteEvent") then returnvalue =
Remote:FireServer(unpack(selected.args))
            elseif Remote:IsA("RemoteFunction") then returnvalue =
Remote:InvokeServer(unpack(selected.args)) end

```

```

        TextLabel.Text = ("Executed
successfully!\n%s"):format(v2s(returnvalue))
        end,function(err) TextLabel.Text = ("Execution
error!\n%s"):format(err) end)
        return
    end
    TextLabel.Text = "Source not found"
end)

newButton("Get Script", function() return "Click to copy calling
script to clipboard\nWARNING: Not super reliable, nil == could not
find" end, function()
    if selected then
        if not selected.Source then selected.Source =
rawget(getfenv(selected.Function),"script") end
        setclipboard(v2s(selected.Source)); TextLabel.Text = "Done!"
    end
end)

newButton("Function Info",function() return "Click to view calling
function information" end, function()
    local func = selected and selected.Function
    if func then
        local typeoffunc = typeof(func)
        if typeoffunc ~= 'string' then
            codebox:setRaw("--[[Generating Function Info please
wait]]"); RunService.Heartbeat:Wait()
            local lclosure = islclosure(func)
            local SourceScript = rawget(getfenv(func),"script")
            local CallingScript = selected.Source or nil
            local info = { info = getinfo(func), constants = lclosure
and deepclone(getconstants(func)) or "N/A --Lua Closure expected got C
Closure", upvalues = deepclone(getupvalues(func)), script = {
SourceScript = SourceScript or 'nil', CallingScript = CallingScript or
'nil' } }
            if configs.advancedinfo then
                local Remote = selected.Remote
                info["advancedinfo"] = { Metamethod =
selected.metamethod, DebugId = { SourceScriptDebugId = SourceScript
and typeof(SourceScript) == "Instance" and OldDebugId(SourceScript) or
"N/A", CallingScriptDebugId = CallingScript and typeof(SourceScript)
== "Instance" and OldDebugId(CallingScript) or "N/A", RemoteDebugId =
OldDebugId(Remote) }, Protos = lclosure and getprotos(func) or "N/A
--Lua Closure expected got C Closure" }
                if Remote:IsA("RemoteFunction") then
info["advancedinfo"]["OnClientInvoke"] = getcallbackmember and
(getcallbackmember(Remote,"OnClientInvoke") or "N/A") or "N/A
--Missing function getcallbackmember"

```

```

                elseif getconnections then
info["advancedinfo"]["OnClientEvents"] = {}; for i,v in next,
getconnections(Remote.OnClientEvent) do
info["advancedinfo"]["OnClientEvents"][i] = { Function = v.Function or
"N/A", State = v.State or "N/A" } end end
                end
                codebox:setRow("--[[Converting table to string please
wait]])")
                selected.Function = v2v({functionInfo = info})
            end
            codebox:setRow("-- Calling function info\n-- Generated by the
SimpleSpy V3 serializer\n\n"..selected.Function)
            TextLabel.Text = "Done! Function info generated by the
SimpleSpy V3 Serializer."
            else TextLabel.Text = "Error! Selected function was not found."
end
end)

newButton("Clr Logs", function() return "Click to clear logs" end,
function()
    TextLabel.Text = "Clearing..."; clear(logs)
    for i,v in next, LogList:GetChildren() do if not
v:IsA("UILayout") then v:Destroy() end end
    codebox:setRow(""); selected = nil; TextLabel.Text = "Logs
cleared!"
end)

newButton("Exclude (i)", function() return "Click to exclude this
Remote.\nExcluding a remote makes SimpleSpy ignore it, but it will
continue to be usable." end, function()
    if selected then blacklist[OldDebugId(selected.Remote)] = true;
TextLabel.Text = "Excluded!" end
end)

newButton("Exclude (n)", function() return "Click to exclude all
remotes with this name.\nExcluding a remote makes SimpleSpy ignore it,
but it will continue to be usable." end, function()
    if selected then blacklist[selected.Name] = true; TextLabel.Text =
"Excluded!" end
end)

newButton("Clr Blacklist", function() return "Click to clear the
blacklist.\nExcluding a remote makes SimpleSpy ignore it, but it will
continue to be usable." end, function()
    blacklist = {}; TextLabel.Text = "Blacklist cleared!"
end)

newButton("Block (i)", function() return "Click to stop this remote

```

```

from firing.\nBlocking a remote won't remove it from SimpleSpy logs,
but it will not continue to fire the server." end, function()
    if selected then blocklist[OldDebugId(selected.Remote)] = true;
TextLabel.Text = "Excluded!" end
end)

```

```

newButton("Block (n)",function() return "Click to stop remotes with
this name from firing.\nBlocking a remote won't remove it from
SimpleSpy logs, but it will not continue to fire the server." end,
function()
    if selected then blocklist[selected.Name] = true; TextLabel.Text =
"Excluded!" end
end)

```

```

newButton("Clr Blocklist", function() return "Click to stop blocking
remotes.\nBlocking a remote won't remove it from SimpleSpy logs, but
it will not continue to fire the server." end, function()
    blocklist = {}; TextLabel.Text = "Blocklist cleared!"
end)

```

```

newButton("Decompile", function() return "Decompile source script"
end,function()
    if decompile then
        if selected and selected.Source then
            local Source = selected.Source
            if not DecompiledScripts[Source] then
                codebox:setRaw("-- [[Decompiling]]")
                xpcall(function()
                    local decompiledsource =
decompile(Source):gsub("-- Decompiled with the Synapse X Lua
decompiler.", "")
                    local Sourcev2s = v2s(Source)
                    if (decompiledsource):find("script") and Sourcev2s
then DecompiledScripts[Source] = ("local script =
%s\n%s"):format(Sourcev2s,decompiledsource) end
                    end,function(err) return codebox:setRaw(("-- [[\nAn
error has occurred\n%s\n]]"):format(err)) end)
                end
                codebox:setRaw(DecompiledScripts[Source] or "--No Source
Found"); TextLabel.Text = "Done!"
            else TextLabel.Text = "Source not found!" end
            else TextLabel.Text = "Missing function (decompile)" end
end)

```

```

newButton("Disable Info", function() return string.format("[%s] Toggle
function info (because it can cause lag in some games)",
configs.funcEnabled and "ENABLED" or "DISABLED") end, function()
    configs.funcEnabled = not configs.funcEnabled; TextLabel.Text =

```

```

string.format("[%s] Toggle function info (because it can cause lag in
some games)", configs.funcEnabled and "ENABLED" or "DISABLED")
end)

newButton("Autoblock", function() return string.format("[%s] [BETA]
Intelligently detects and excludes spammy remote calls from logs",
configs.autoblock and "ENABLED" or "DISABLED") end, function()
    configs.autoblock = not configs.autoblock; TextLabel.Text =
string.format("[%s] [BETA] Intelligently detects and excludes spammy
remote calls from logs", configs.autoblock and "ENABLED" or
"DISABLED")
    history = {}; excluding = {}
end)

newButton("Logcheckcaller",function() return ("[%s] Log remotes fired
by the client"):format(configs.logcheckcaller and "ENABLED" or
"DISABLED") end, function()
    configs.logcheckcaller = not configs.logcheckcaller;
TextLabel.Text = ("[%s] Log remotes fired by the
client"):format(configs.logcheckcaller and "ENABLED" or "DISABLED")
end)

newButton("Advanced Info",function() return ("[%s] Display more
remoteinfo"):format(configs.advancedinfo and "ENABLED" or "DISABLED")
end, function()
    configs.advancedinfo = not configs.advancedinfo; TextLabel.Text =
("[%s] Display more remoteinfo"):format(configs.advancedinfo and
"ENABLED" or "DISABLED")
end)

newButton("Join Discord",function() return "Joins The Simple Spy
Discord" end, function()
    setclipboard("https://discord.com/invite/AWS6ez9"); TextLabel.Text
= "Copied invite to your clipboard"
    if request then request({Url =
'http://127.0.0.1:6463/rpc?v=1',Method = 'POST',Headers =
{'Content-Type' = 'application/json', Origin =
'https://discord.com'},Body = http.JSONEncode({cmd =
'INVITE_BROWSER',nonce = http.GenerateGUID(false),args = {code =
'AWS6ez9'}})}) end
end)

if configs.supersecretdevtoggle then
    newButton("Load SSV2.2",function() return "Load's Simple Spy V2.2"
end, function()
loadstring(game:HttpGet("https://raw.githubusercontent.com/exxtremestu
ffs/SimpleSpySource/master/SimpleSpy.lua"))() end)
    newButton("Load SSV3",function() return "Load's Simple Spy V3"

```

```

end, function()
loadstring(game:HttpGet("https://raw.githubusercontent.com/78n/SimpleSpy/main/SimpleSpySource.lua"))() end)
    local SuperSecretFolder = Create("Folder",{Parent = SimpleSpy3})
    newButton("SUPER SECRET BUTTON",function() return "You dont need a
discription you already know what it does" end, function()
        SuperSecretFolder:ClearAllChildren(); local random =
listfiles("Music")
        local NotSound = Create("Sound",{Parent =
SuperSecretFolder,Looped = false,Volume = math.random(1,5),SoundId =
getsynasset(random[math.random(1,#random)])})
        NotSound:Play()
    end)
end

```